Laboratório Funcional · Aula 12

Miguel Falcão Costacurta · RA 240113022 · Turma B
Gerado em 16/10/2026, 20:29:42 · tempo registrado no cronômetro: 30:39

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Pura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

def agora(): return datetime.now() - Retorna valores diferentes dependendo do momento em que é chamada.
def sorteia(): return random.randint(1, 6) - Retorna valores aleatórios, não garante o mesmo resultado em chamadas sucessivas.
def poe(lista, x): lista.append(x); return lista - Modifica variáveis externas.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

1: A-1 A-2
4: 14
7: [7, 3, 9] [3, 7, 9]

Saídas (original e com compor(g, f))

```
Antes: 4: 14
Depois: 4: 18
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

Ele devolve valores diferentes por conta do contador global dentro da função.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

a diferença ocorre por conta da ordem que é executada a função, fazendo 4+2 e depois 3x6 temos 18. Já no outro caso 3x4 = 12 e depois 12 + 2.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma nova lista ordenada sem alterar a original, seguindo melhor o estilo funcional. Já notas.sort() altera a própria lista, causando um efeito colateral.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A B C

Previsão: (cdr '(A B C))

(A B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
Backtrace:
           5 (primitive-load "/box/main.scm")
In ice-9/eval.scm:
   191:35  4 (_ #f)
   191:35  3 (_ #f)
   196:27  2 (_ #f)
   223:20  1 (proc #<directory (guile-user) 98b572c9c80>)
In unknown file:
           0 (%resolve-variable (7 . A) #<directory (guile-user) 98b…>)

ERROR: In procedure %resolve-variable:
Unbound variable: A
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

o ' serve para o código tratar como dados o que vem na frente, sem o ' ele tenta fazer a chamada de uma função.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
(car (cdr (cdr '(A B C))))
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

n = 3: comum em 6 passos, até 2 multiplicações esperando (expressão de até 24 caracteres); em cauda em 5 passos, nada esperando (até 16 caracteres).
n = 4: comum em 8 passos, até 3 multiplicações esperando (expressão de até 30 caracteres); em cauda em 6 passos, nada esperando (até 17 caracteres).

n = 6: comum em 12 passos, até 5 multiplicações esperando (expressão de até 42 caracteres); em cauda em 8 passos, nada esperando (até 18 caracteres).

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

Scheme: provavelmente funcionará se o interpretador otimizar a recursão de cauda, pois pode reutilizar a pilha de chamadas.
Python: provavelmente dará erro por conta da profundidade.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
# Prática 3b - A mesma recursão em cauda, agora em Python
# Cole em https://onecompiler.com/python
def conta_ate(n, acc):
    if n == 0:
        return acc
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação

print(conta_ate(500, 0))
print(conta_ate(1_000_000, 0))
Exit 1
500
Traceback (most recent call last):
  File "/box/main.py", line 9, in <module>
    print(conta_ate(1_000_000, 0))
          ^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/box/main.py", line 6, in conta_ate
    return conta_ate(n - 1, acc + 1)     # a chamada é a última operação
           ^^^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, crescem as operações pendentes na pilha de chamadas, porque as multiplicações precisam esperar o retorno das chamadas seguintes. Já na em cauda, como ela não tem operações pendentes não cresce dessa forma.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme otimiza chamadas em cauda, reutilizando o espaço da pilha. Já o Python acumula chamadas recursivas até atingir o limite de recursão, gerando RecursionError.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 3  ok
soma-quadrados: 14  ok
pares: (2 4 6)  ok
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
  (if (null? lis)
      0
      (+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
  (if (null? lis)
      0
      (+ (* (car lis) (car lis))
         (soma-quadrados (cdr lis)))))

(define (pares lis)
  (cond ((null? lis) '())
        ((even? (car lis))
         (cons (car lis) (pares (cdr lis))))
        (else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender a recursão. Usei null? para verificar se a lista está vazia, car para acessar o primeiro elemento e cdr para continuar com o restante da lista.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime os cinco primeiros quadrados da lista: [0,1,4,9,16]. O outro não sei, 0 talvez.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42

Time limit exceeded
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell usa avaliação preguiçosa, calculando apenas os valores necessários. Assim, consegue imprimir parte de uma lista infinita sem calcular tudo.

A função primeiro 42 (div 1 0) não dá erro porque o segundo parâmetro não é usado e, portanto, a divisão por zero nunca é calculada.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

member 16 squares termina porque encontra o número 16. Já member 17 squares nunca termina porque a lista de quadrados é infinita e 17 não aparece nela.

```
member b [] = False
member b (a:x)
 | a > b    = False
 | a == b   = True
 | otherwise = member b x
```

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

15

5 10 15

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
System.out.println(caros);
        System.out.println(produtos.stream().map(p →
p.nome().toUpperCase()).sorted().toList());
    }
}
1.3s
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

Isso acontece por causa de um closure (fechamento). A função devolvida mantém acesso à variável n, mesmo depois que somador(10) termina. Assim, mais10(5) soma 5 + 10 e retorna 15.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

compose executa g antes de f, já o andThen executa f antes de g. segundo exemplo.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
int soma = nums.stream()
    .filter(n → n % 2 == 0)
    .mapToInt(n → n * n)
    .sum();

System.out.println(soma);
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244
ANA, CARLA, DAVI
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão com laço costuma ser mais fácil de depurar, pois permite verificar os valores a cada etapa. A versão funcional é mais curta e, em operações simples, pode ser mais fácil de ler. Isso depende da complexidade do código e da familiaridade de quem o lê.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em C é mais fácil de entender para quem está acostumado com laços e variáveis. A versão em Haskell é mais curta e expressa diretamente as operações. Aceitaria o custo extra em sistemas de análise de dados, mas não em sistemas de tempo real que exigem alta velocidade.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que o Haskell consegue trabalhar com listas infinitas calculando apenas os elementos necessários.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro está em b <- h, que deveria ser b <= h. O correto é:

```
sort [] = []
sort (h:t) = sort [b | b <- t, b <= h]
        ++ [h] ++
        sort [b | b <- t, b > h]
```

O compilador reclama porque h é um elemento, mas b <- h exige uma lista para percorrer. A correção usa b <= h como condição de comparação.